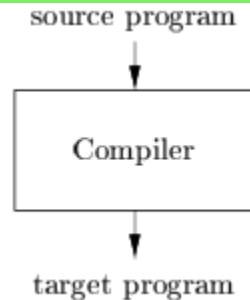


Chapter-01 (Introduction) + Introduction to Compiler slide

1. What does language translator means? Give some example of source language, translation and target language.

Translator read source language and translate it into target language.



- **Source program:** High level language programming
Ex: 'c' programming language
- **Translator:** Translate The Source program
Ex: 'Compiler', Interpreter.
- **Target program:** output of the translation process
Ex: Machine code, Intermediate code.

2. What does compiler mean? list the challenges of building a compiler.

A **compiler** is a program that translates a source program written in some *high-level programming language (such as Java) into machine code* for some computer architecture (such as the Intel Pentium architecture).

Challenges of building a compiler:

Many variations:

- Many programming languages (eg, Java, C++, Scala)
- Many programming paradigms (eg, object-oriented, functional, logic)
- Many computer architectures (eg, x86, MIPS, ARM, alpha, PowerPC)
- Many operating systems (eg, Linux, OS X, Microsoft Windows, Android, iOS)

Qualities of a compiler (in order of importance):

- It must generate correct machine code
- The generated machine code must run fast
- The compiler must be portable (ie, modular, supporting separate compilation)
- It must print good diagnostics and error messages

Building a compiler requires knowledge of programming languages (parameter passing, variable scoping, memory allocation, etc)

- Theory (automata, context-free languages, etc)
- Algorithms and data structures (hash tables, graph algorithms, dynamic programming, etc)
- Computer architecture (assembly programming)
- Software engineering.

3. How do you distinguish between a compiler and an interpreter? What advantages are there to a language-processing system in which the compiler produces assembly language rather than machine language?

Distinguish between **compiler** and an **interpreter**:

Aspect	Compiler	Interpreter
Execution	Converts entire code to machine language at once.	Executes code line by line.
Speed	Faster after compilation (single-time process)	Slower (line-by-line execution each time).
Error Detection	Shows all errors after compilation.	Detects errors line by line while running.
Output	Creates a standalone executable file.	No separate file; executes directly.
Example	C, C++	Python, JavaScript.

A compiler that produces assembly language instead of machine language has these advantages:

1. **Readability:** Assembly code is easier for humans to read and debug compared to machine code (binary).
 2. **Portability:** It can be assembled for different machines by using specific assemblers.
 3. **Flexibility:** Developers can optimize or modify the assembly code before converting it to machine language.
 4. **Simpler Compiler Design:** Generating assembly is easier than directly producing machine code, especially for complex CPUs.
4. Show the steps of a language-processing system. How do you distinguish between a compiler and an interpreter? (Already noted in question 3)
- **Preprocessor:** It processes the source code to handle directives like `#include` and `#define`, preparing it for compilation.
 - **Compiler:** It translates the modified source code into assembly language.
 - **Assembler:** It converts the assembly language code into machine-level object code.
 - **Linker/Loader:** It combines object code with library files, resolves references, and loads the final executable program into memory.

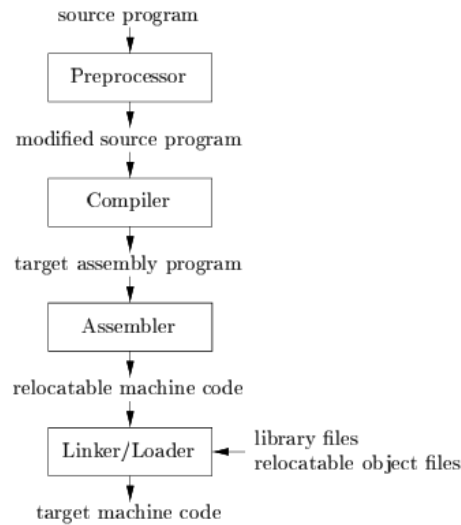


Figure 1.5: A language-processing system

5. Write the responsibilities of preprocessor, linker and loader

❖ **Preprocessor:** Prepares the code for compilation by handling `#include`, `#define`, and removing comments.

Example:

- ✓ `#define PI 3.14`
- ✓ `#include <stdio.h>`
- ✓ Removes comments: `// This is a comment`

❖ **Linker:** Combines object files and links libraries to create the final executable.

Example:

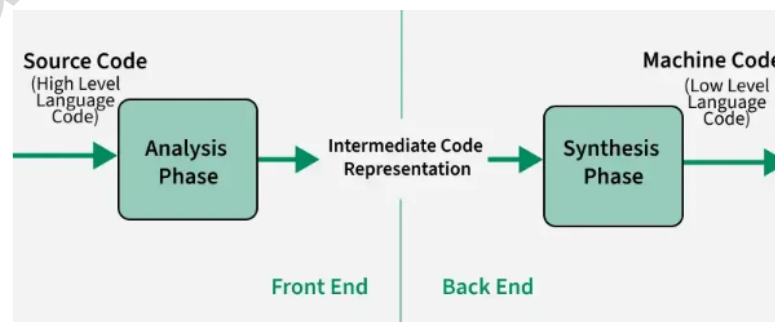
- ✓ Links `main.o` and `math.o` into one executable.
- ✓ Adds the code for `printf()` from the standard library.

❖ **Loader:** Loads the executable into memory and starts the program.

Example: When you run `my_program.exe`, the loader puts it into memory and begins execution.

6. Illustrate the typical decomposition of a compiler into phases (See next question solution). Identify the front end and the back-end parts in the structure of the compiler.

The front end (Analysis Phase) reads the source code and converts it into an intermediate form, while the back end (Synthesis Phase) takes that intermediate code and generates the final machine code.



7. Illustrate the typical decomposition of a compiler into phases. Suppose a source program contains the assignment statement: `position = initial + rate * 60`, show the representation of the assignment statement using the different phases in the compiler.

Here's the mnemonic sentence written clearly: **"Lazy Silly Students Invent Many Crazy Machines".**

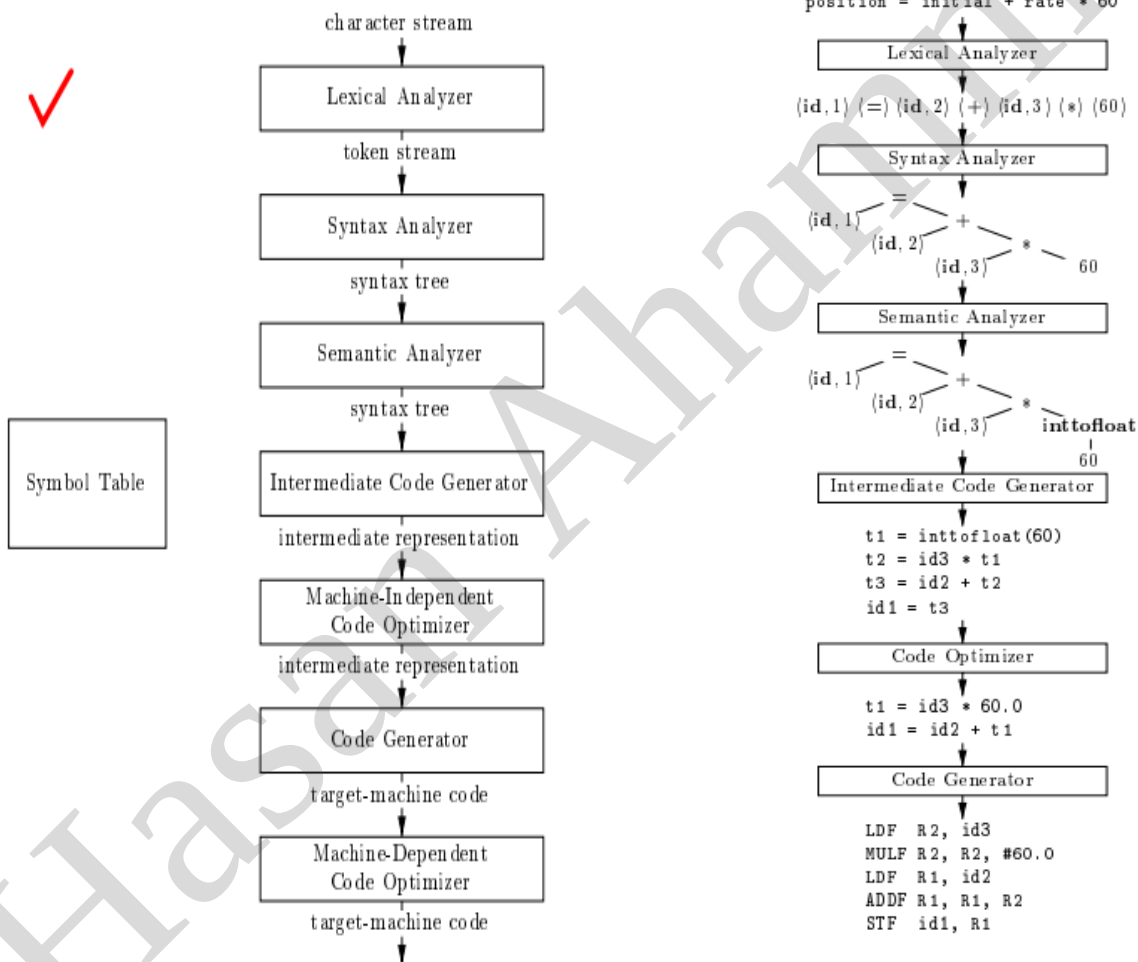
This funny and rhyming sentence can help remember the main phases of a compiler in a sequence.

Meaning:

- ✓ Lazy → Lexical Analyzer
- ✓ Silly → Syntax Analyzer
- ✓ Students → Semantic Analyzer

- ✓ Invent → Intermediate Code Generator
- ✓ Many → Machine-Independent Code Optimizer

- ✓ Crazy → Code Generator
- ✓ Machines → Machine-Dependent Code Optimizer



8. Model of a compiler front end:

Explanation of the Compiler Front End:

- **Source Program** → Input code (e.g., `position = initial + rate * 60`).
- **Lexical Analyzer (Scanner)** → Breaks code into tokens (e.g., `position`, `=`, `initial`, `+`, `rate`, `*`, `60`).
- **Parser** → Checks grammar and builds a syntax tree (hierarchical structure of operations).
- **Intermediate Code Generator** → Converts syntax tree into three-address code (simplified instructions like `temp1 = rate * 60`, `temp2 = initial + temp1`, `position = temp2`).
- **Symbol Table** → Maintains variable info (names, types, memory locations) throughout compilation.

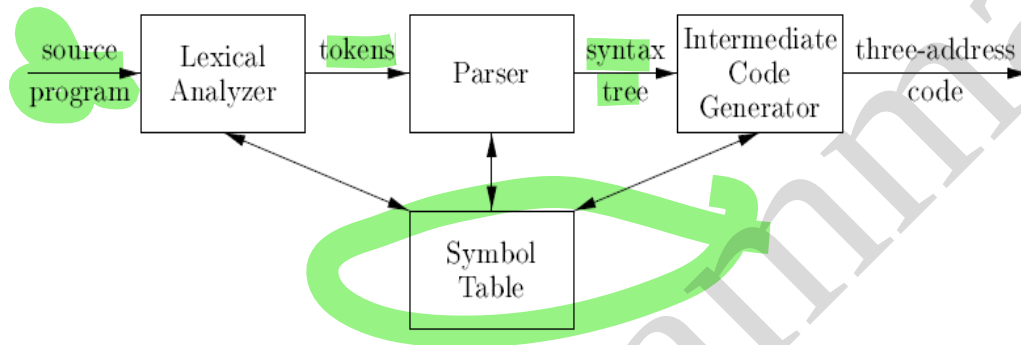


Figure 2.3: A model of a compiler front end